

Capítulo 1. Introdução e Conceitos básicos

1. Linguagens
2. Gramáticas
3. Autômatos

1.1. Linguagens

1. Alfabeto, $\Sigma = \{\text{símbolos}\}$, conjunto finito de símbolos (letras, algarismos, ...)

- $\{a, b\}$,
- $\{0, 1, 2, \dots, 9\}$, conjunto dos algarismos árabes
- $\{0, 1\}$, alfabeto binário
- $\{[, (,),]\}$,
- $\{\#, \$, \%, \&\}$
- $\{a, b, c, \dots, z\}$, o alfabeto romano
- $\{\text{copos, facas, garfos, pratos, colheres, tachos}\}$
- $\emptyset$ é um alfabeto mas não é útil.

Qualquer objeto pode pertencer a um alfabeto.

Não são alfabetos (por quê?)

- $\mathbb{N}$ (conjunto dos números naturais)
- $\{a, b, aa, ab, ba, bb, aaa, \dots\}$

1.1. Linguagens - Alfabeto

Exp: Alfabeto: linguagem de programação

Alfabeto de uma linguagem de programação como Pascal

- o conjunto de todos os símbolos usados nos programas
 - * letras
 - * dígitos
 - * caracteres especiais como ">", "/", etc
 - * espaço ou "branco"

Alfabeto binário {a, b}

- domínio de valores de um bit
- analogia com a representação interna dos computadores reais
- poucos símbolos: simplifica as diversas abordagens desenvolvidas

2. Palavra ou Cadeia (*string*), w

$w = \{\text{sequência finita de símbolos justapostos do alfabeto; pode ser vazia}\}$

$$\Sigma = \{a, b\},$$

$$w = a, w = ab, w = abbaba$$

3. Concatenação de cadeias w, v

$$w = a_1 a_2 a_3 \dots a_n \quad v = b_1 b_2 b_3 \dots b_m$$

concatenação de w e v

$$wv = a_1 a_2 a_3 \dots a_n b_1 b_2 b_3 \dots b_m$$

concatenação de v e w ,

$$vw = b_1 b_2 b_3 \dots b_m a_1 a_2 a_3 \dots a_n$$

Propriedades

- **Elemento neutro:** $\varepsilon w = w = w \varepsilon$
- **Associativa:** $v(wt) = (vw)t$
- Associatividade – parênteses podem ser omitidos: $vw t$

4. Cadeia reversa , $w^R = a_n \dots a_3 a_2 a_1$

5. Comprimento de uma cadeia, $|w|$:

número de símbolos da cadeia

$$|abcb|=4$$

6. Cadeia vazia, λ

cadeia sem qualquer símbolo, $|\lambda|=0$

$$\lambda w = w \lambda = w, \text{ para todo } w$$

ε - cadeia vazia ou palavra vazia

7. Palíndromos (*palindromes*) em Σ : cadeias tais que

$$w = w^R$$

8. Subcadeia de uma cadeia

w

qualquer cadeia composta por caracteres sucessivos de w

$w=uv$, u é o **prefixo** e v o **sufixo** de w

Exemplo: $w=abbab$

Subcadeias de w : $a, b, bb, bba, \dots$

Prefixos de w : $\{ \ , a, ab, abb, abba, abbab \}$

Sufixos de w : $\{ abbab, bbab, bab, ab, b, \lambda \}$

8. Subcadeia de uma cadeia w

Def: Prefixo, sufixo, subpalavra

Prefixo (Sufixo)

- qualquer sequência inicial (final) de símbolos da palavra

Subpalavra

- qualquer sequência de símbolos contíguos da palavra

Exp: Palavra, prefixo, sufixo, subpalavra

abcb palavra sobre o alfabeto $\{a, b, c\}$

- ϵ , **a**, **ab**, **abc**, **abcb** são todos os prefixos
- ϵ , **b**, **cb**, **bcb**, **abcb** são todos os sufixos
- qualquer prefixo ou sufixo é uma subpalavra

Em uma linguagem de programação como Java, C

uma palavra é um programa

8

9. Potências de uma cadeia w^n (Concatenação de palavras)

cadeia obtida pela (auto)concatenação n vezes da cadeia w , $w^0 = \lambda$, para todo o w .

w palavra e a símbolo

- $w^3 = w w w$
- $w^1 = w$
- $a^5 = aaaaa$
- $a^n = aaa...a$ (o símbolo a repetido n vezes)

9. Potências de uma cadeia w^n (Concatenação de palavras)
cadeia obtida pela (auto)concatenação n vezes da cadeia w , $w^0 = \lambda$, para todo o w .

10. Σ^* : conjunto de todas as palavras que se podem obter pela concatenação de zero ou mais símbolos de Σ .
Contém sempre a cadeia nula λ ou ϵ (palavra vazia)

11. $\Sigma^+ = \Sigma^* - \{\lambda\}$, i.e., exclui a cadeia vazia

quer Σ^* quer Σ^+ são sempre infinitas, mesmo quando Σ é finito.

12. Linguagem L : qualquer subconjunto de Σ^*

13. Frase (sentença ou palavra) : qualquer cadeia/palavra/string na linguagem L .

Exemplo 1:

$$\Sigma = \{a, b\}$$

$\Sigma^* = \{ \varepsilon \text{ ou } \lambda, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, \dots \}$,
todas as combinações possíveis dos símbolos do alfabeto,
incluindo a cadeia vazia.

$$\Sigma^+ = \{ a, b, aa, ab, ba, bb, aaa, \dots \}$$

O conjunto $L = \{a, ab, abb\}$ é uma linguagem em Σ .

É uma linguagem **finita**.

Exemplo 2.

O conjunto $L = \{a^n b^n : n \geq 0\}$ é uma linguagem em Σ .

É uma linguagem **infinita**.

$ab, aabb, aaabbb, \dots$ pertencem a L

$aba, abb, aaababbab$, não pertencem a L .

Uma linguagem é um **conjunto** !

Portanto aplicam-se-lhe todas as operações sobre conjuntos:

14. União: $L_1 \cup L_2 = \{w: w \in L_1 \text{ ou } w \in L_2\}$

15. Intersecção: $L_1 \cap L_2 = \{w: w \in L_1 \text{ e } w \in L_2\}$

16. Diferença de duas linguagens: $L_1 - L_2 = \{w: w \in L_1 \text{ e } w \notin L_2\}$

17. Complemento de uma linguagem: $\text{Compl}(L) = \Sigma^* - L$

18. Reverso de uma linguagem: $L^R = \{w^R : w \in L\}$

19. Concatenação de duas linguagens, $L_1 \bullet L_2$, $L_1 L_2$

$$L_1 \bullet L_2 = L_1 L_2 = \{uv : u \in L_1, v \in L_2\}$$

$$L_1 \bullet (L_2 \cup L_3) = L_1 \bullet L_2 \cup L_1 \bullet L_3$$

(distributividade da concatenação em ordem à união)

$$L_1 \cup (L_2 \bullet L_3) \neq L_1 \cup L_2 \bullet L_1 \cup L_3$$

em geral a união não é distributiva sobre a concatenação.

$$L \bullet \emptyset = \emptyset$$

$$L \bullet \lambda = L$$

20. Potência de uma linguagem: concatenação com ela própria:

$$L^0 = \{\lambda\}, \quad L^1 = L, \quad L^2 = LL, \quad L^3 = LLL, \quad \dots$$

21. Fecho-estrela (*star-closure*, *Kleene star*, *Kleene closure*) L^* :

todas as cadeias que se podem obter por concatenação da linguagem com ela própria, incluindo sempre a cadeia vazia:

$$L^* : L^0 \cup L^1 \cup L^2 \cup L^3 \cup \dots$$

Fecho-positivo (*positive closure*) L^+ : todas as cadeias que se podem obter por concatenação da linguagem com ela própria; não contém a cadeia vazia se L não contém ,

$$L^+ : L^1 \cup L^2 \cup L^3 \cup \dots = L^* - \{\lambda\} \quad (\text{se } \lambda \notin L)$$

Exemplo : $L = \{a^n b^n, n \geq 0\}$

$L = \{\varepsilon, ab, aabb, aaabbb, aaaabbbb, \dots\}$ (note-se que $\varepsilon \in L$)

$L^2 = LL = \{ \varepsilon, ab, aabb, aaabbb, aaaabbbb, \dots$
 $abab, abaabb, abaaabbb, abaaaabbbb, \dots$
 $aabbab, aabbaabb, aabbbaabbb, \dots \}$

$= \{a^n b^n a^m b^m, n \geq 0, m \geq 0\}$, n e m são independentes.

Reversa de L : $L^R = \{b^n a^n, n \geq 0\}$

Complemento de L : ???

Fecho estrela L^* : ??? L^+ : ???

Será a notação de conjuntos adequada para especificar linguagens ?

Linguagem Formal

Def: Linguagem formal

Linguagem formal ou simplesmente linguagem L sobre um alfabeto Σ

$$L \subseteq \Sigma^*$$

Exp: Linguagem formal

$\emptyset$ e $\{\varepsilon\}$ são linguagens sobre qualquer alfabeto

$$\emptyset \neq \{\varepsilon\}$$

Σ^* e Σ^+ são linguagens sobre um Σ qualquer

$$\Sigma^* \neq \Sigma^+$$

Conjunto de **palíndromos** sobre $\Sigma = \{a, b\}$

Linguagem Formal

Conjunto de **palíndromos** sobre $\Sigma = \{a, b\}$

$\epsilon, a, b, aa, bb, aaa, aba, bab, bbb, aaaa, \dots$

Ex: Conjunto de todas as linguagens sobre um alfabeto

Conjunto das partes de $\Sigma^* = 2^{\Sigma^*}$

Ex: Linguagem formal: linguagem de programação

Linguagem de programação como C, C++, Java, ...

conjunto de todos os programas (palavras) da linguagem

1.2. Gramáticas

Uma ferramenta para estudar matematicamente linguagens, muito poderosa, que ultrapassa as limitações da notação de conjuntos anterior.

Linguagem de programação

- definida pelo conjunto de todos os programas (palavras)

Linguagem de propósitos gerais como C, C++,Java

- conjunto de todos os programas é *infinito*
- não é definição adequada para implementação em computador

Formalismo de gramática

- uma maneira de especificar de forma *finita* linguagens (eventualmente) *infinitas*

1.2. Gramáticas

Gramática é, basicamente:

- conjunto *finito* de regras
- quando aplicadas sucessivamente, geram palavras
- conjunto de todas as palavras geradas por uma gramática
 - * define a linguagem

Gramáticas para linguagens naturais como Português

- as mesmas que as usadas para linguagens artificiais como C, C++, Java

Gramáticas também são usadas para definir semântica

- entretanto, em geral, são usados outros formalismos

Exemplo da gramática da língua

portuguesa:

Regras de produção de uma frase (ou oração) simples, em versão simplificada e incompleta, para fins ilustrativos :

⟨frase⟩ → ⟨sintagma nominal⟩ ⟨sintagma verbal⟩
⟨sintagma nominal⟩ → ⟨determinante⟩ ⟨nome⟩ | ⟨vazio⟩
⟨sintagma verbal⟩ → ⟨ verbo⟩ ⟨sintagma nominal⟩
⟨determinante⟩ → ⟨artigo⟩ | ⟨deícticos⟩ | ⟨vazio⟩
⟨verbo⟩ → estuda | ama | compra | dorme|chove
⟨artigo⟩ → o | a |um | uma|<vazio>
⟨deícticos⟩ → este | esse | aquele | meu | teu | seu |<vazio>
⟨nome⟩ → Luís | Antónia | Isabel | livro | gelado |

O Luís compra um livro : como derivar ? (*parsing*)

(ver tb Gramática da Língua Portuguesa, Mateus, M., A. M.Brito, I. Duarte, I. H Faria, Caminho Série Linguística, 1989, p. 210

Verificar se as seguintes frases obedecem a essas regras de produção:

- (i) O João vai ao cinema
- (ii) Chove
- (iii) Um por todos, todos por um !

Este exemplo mostra como uma frase (ou oração), conceito geral e complexo, pode ser definida à custa de elementos simples (decomposição da complexidade).

Começa-se no conceito mais complexo (frase), e reduz-se sucessivamente até se obterem os elementos irreduzíveis com que se constrói a linguagem.

A generalização deste princípio leva-nos à definição de gramáticas formais.

Definição 1.1.

Uma gramática G é definida por um quarteto

$$G = (V, T, S, P) \text{ OU } G = (V, T, P, S)$$

Gramática de Chomsky, Gramática irrestrita ou somente gramática.

V : variáveis, conjunto finito de objectos, não vazio

T : símbolos Terminais, conjunto finito de objectos, não vazio

$S \in V$, variável de *inicialização* (Semente, *Start*, *Axioma*)

P : um conjunto finito de Produções

$$P: (V \cup T)^+ \rightarrow (V \cup T)^*$$

V e T são disjuntos

As regras de produção

- o cerne da gramática, pois
- é através delas que se forma uma cadeia
- e que uma cadeia se transforma noutra cadeia

Por isso elas **definem uma linguagem** associada à gramática .

Uma linguagem pode ser produzida por várias gramáticas, mas uma gramática produz uma e só uma linguagem.

Todas as regras de produção são da forma

$$x \rightarrow y \text{ ou } (x, y)$$

$$x \in (V \cup T)^+,$$

cadeia de variáveis e símbolos terminais, excluindo a cadeia vazia

$$y \in (V \cup T)^*,$$

cadeia de variáveis e símbolos terminais, podendo incluir a cadeia vazia.

- dada

$$w = u x v$$

- dada

$$x \rightarrow y$$

$$z = u y v$$

por substituição de x por y em w , obtém-se a cadeia z

- w **produz** z , ou z **é produzida** por w .

$$w \Rightarrow z$$

As regras de produção podem aplicar-se por **qualquer ordem**
e **tantas vezes quantas as necessárias !!!**

Se

$$w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n$$

diz-se que w_1 *produz* w_n ; em escrita mais compacta

$$w_1 \overset{*}{\Rightarrow} w_n$$

em que $*$ indica que para derivar w_n de w_1 são necessários um número não especificado de passos (incluindo zero passos, como em $w_1 \overset{*}{\Rightarrow} w_1$).

- As regras de produção podem ser aplicadas numa **ordem qualquer** e um **número qualquer** de vezes.
- Por cada ordem e por cada número, produzem uma cadeia terminal.

O conjunto de todas as cadeias terminais que se podem obter pela gramática, forma a **linguagem gerada pela gramática**, $L(G)$

$$G = (V, T, S, P)$$

$$L(G) = \{ w \in T^* : S \xRightarrow{+} w \}$$

Se $w \in L(G)$, então a sequência

$$S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \dots \Rightarrow w_n \Rightarrow w$$

é uma **derivação** da frase w .

Formas sentenciais da derivação de w são as cadeias

$$S, w_1, w_2, \dots, w_n$$

As formas sentenciais contêm variáveis e símbolos terminais (em alguns casos só contêm variáveis).

Uma derivação de uma palavra é:

- substituição de uma subpalavra
- de acordo com uma regra de produção

Sucessivos passos de derivação

- $\Rightarrow^*$ fecho transitivo e reflexivo da relação $\Rightarrow$
 - * zero ou mais passos de derivações sucessivos
- $\Rightarrow^+$ fecho transitivo da relação $\Rightarrow$
 - * um ou mais passos de derivações sucessivos
- $\Rightarrow^i$
 - * exatos i passos de derivações sucessivos (i natural)

Gramática, derivação, linguagem gerada: números naturais :

$$G = (V, T, P, N)$$

- $V = \{ N, D \}$
- $T = \{ 0, 1, 2, \dots, 9 \}$
- $P = \{ N \rightarrow D, N \rightarrow DN, D \rightarrow 0 \mid 1 \mid \dots \mid 9 \}$

Gera, sintaticamente, o conjunto dos números naturais

- se distinguem os zeros à esquerda
- exemplo: 123 de 0123

Ex: Gramática, derivação, linguagem gerada: números naturais :

Uma derivação do número 243

- $N \Rightarrow$
- $DN \Rightarrow$
- $2N \Rightarrow$
- $2DN \Rightarrow$
- $24N \Rightarrow$
- $24D \Rightarrow$
- 243

$N \rightarrow DN$

$D \rightarrow 2$

$N \rightarrow DN$

$D \rightarrow 4$

$N \rightarrow D$

$D \rightarrow 3$

Ex: Gramática, derivação, linguagem gerada:
números naturais :

Portanto

- $S \Rightarrow^* 243$
- $S \Rightarrow^+ 243$
- $S \Rightarrow^6 243$

Interpretação indutiva da gramática

- *Base de indução*: todo dígito é natural
- *Passo de indução*: se n é natural, então a concatenação com qualquer dígito também é natural

Ex: Gramática, derivação, linguagem gerada:
palavra duplicada :

$$G = (\{ S, X, Y, A, B, F \}, \{ a, b \}, P, S)$$

na qual:

- $P = \{ S \rightarrow XY,$
- $X \rightarrow XaA \mid XbB \mid F$
- $Aa \rightarrow aA, Ab \rightarrow bA, AY \rightarrow Ya,$
- $Ba \rightarrow aB, Bb \rightarrow bB, BY \rightarrow Yb,$
- $Fa \rightarrow aF, Fb \rightarrow bF, FY \rightarrow \varepsilon \}$

gera a linguagem

$$\{ ww \mid w \text{ é palavra de } \{ a, b \}^* \}$$

Ex: Gramática, derivação, linguagem gerada: palavra duplicada :

Derivação de **baba**

- $S \Rightarrow$
- $XY \Rightarrow$
- $XaAY \Rightarrow$
- $XaYa \Rightarrow$
- $XbBaYa \Rightarrow$
- $XbaBYa \Rightarrow$
- $XbaYba \Rightarrow$
- $FbaYba \Rightarrow$
- $bFaYba \Rightarrow$
- $baFYba \Rightarrow$
- **baba**

$S \rightarrow XY$
 $X \rightarrow XaA$
 $AY \rightarrow Ya$
 $X \rightarrow XbB$
 $Ba \rightarrow aB$
 $BY \rightarrow Yb$
 $X \rightarrow F$
 $Fb \rightarrow bF$
 $Fa \rightarrow aF$
 $FY \rightarrow \varepsilon$

Existe mais alguma derivação de **baba**?

Definição de Gramáticas Equivalentes

Def: Gramáticas equivalentes

G_1 e G_2 são gramáticas equivalentes se e somente se

$$\text{GERA}(G_1) = \text{GERA}(G_2)$$

Convenções

- $A, B, C, \dots, S, T$ para símbolos variáveis
- $a, b, c, \dots, s, t$ para símbolos terminais
- u, v, w, x, y, z para palavras de símbolos terminais
- $\alpha, \beta, \dots$ para palavras de símbolos variáveis ou terminais

Exemplo:

$$G = (\{S\}, \{a, b\}, S, P)$$

Variáveis: S

Símbolos terminais: a, b .

Variável de inicialização: S

Produções:

$$P_1: S \rightarrow a S b$$

$$P_2: S \rightarrow \epsilon \quad \text{ou}$$

$$S \rightarrow \lambda$$

$$S \Rightarrow a S b \Rightarrow a a S b b \Rightarrow a a a S b b b \Rightarrow \dots$$

Em qualquer altura se pode aplicar a produção P_2 . Depois da

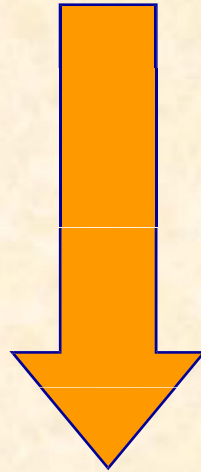
- 1ª produção obtém-se $a b$
- 2ª produção obtém-se $a a b b$
- n-ésima produção $a a a \dots a b b b \dots b = a^n b^n$

Se aplicarmos primeiro a produção P_2 , obtém-se a cadeia vazia

$$G = (\{S\}, \{a, b\}, S, P)$$

$$P_1: S \rightarrow aSb$$

$$P_2: S \rightarrow \lambda$$



$$L(G) = \{a^n b^n, n \geq 0\}$$

Exemplo: Encontrar uma gramática que gere a linguagem

$$L = \{ a^{n+1}b^n, n \geq 0 \}$$

Relativamente ao exemplo anterior, é necessário gerar um a adicional. Pode-se gerá-lo em primeiro lugar, aplicando depois regras de produção como no caso anterior

$$\begin{array}{l} P_1: S \rightarrow a S \\ P_2: S \rightarrow a S b \\ P_3: S \rightarrow \lambda \end{array}$$

$$\begin{array}{l} \text{Antes } P_1: S \rightarrow a S b \\ P_2: S \rightarrow \lambda \end{array}$$

$$G = (\{S\}, \{a,b\}, S, P)$$

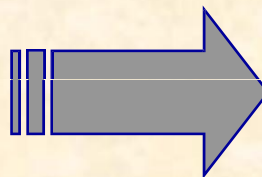
Aplicando P_1, P_1, P_2, P_3 ,

$$S \Rightarrow a S \Rightarrow a a S \Rightarrow a a a S b \Rightarrow a a a b$$

i.e. a^3b , que não faz parte da linguagem (porquê ?)

Para que isto não aconteça, P_1 só se pode aplicar uma vez.
Para isso introduz-se uma variável adicional, A , e definem-se as produções de modo que S só possa produzir uma vez :

$P_1: S \rightarrow a S$
 $P_2: S \rightarrow a S b$
 $P_3: S \rightarrow \lambda$



$P_1: S \rightarrow a A$
 $P_2: A \rightarrow a A b$
 $P_3: A \rightarrow \lambda$

$G = (\{A, S\}, \{a, b\}, S, P)$

em que S é a variável de inicialização. Resulta a gramática G .

Para demonstrar que uma dada linguagem L é gerada por uma certa gramática G , tem que se mostrar que:

- (i) toda a cadeia $w \in L$ pode ser derivada a partir de S usando as produções P da gramática G ,
- (ii) qualquer cadeia gerada pela gramática G pertence à linguagem L .

Uma gramática de palíndromos (PAL) sobre um alfabeto Σ

Pode-se definir um palíndromo pelas três regras seguintes:

1. $\lambda \in \text{PAL}$

2. Para todo o carácter $a \in \Sigma$, $a \in \text{PAL}$

3. Para toda a $w \in \text{PAL}$, e todo $a \in \Sigma$, $awa \in \text{PAL}$

Uma w só pertence a PAL se puder ser gerada por estas 3 regras

Exemplo $\Sigma = \{ a, b \}$

1. $\lambda, a, b \in \text{PAL}$;
2. Para qualquer $w \in \text{PAL}$, awa e $bwb \in \text{PAL}$

Por exemplo:

$b \in \text{PAL}$,	aba	pela regra 1
$\in \text{PAL}$,		pela regra 2
$babab \in \text{PAL}$		pela regra 2
$abababa \in \text{PAL}$		pela regra 2

Uma gramática para esta linguagem

- | | | |
|--|-----------|--|
| 1. $S \rightarrow \lambda$ ou a ou b | (regra 1) | 1. $S \rightarrow \lambda \mid a \mid b$ |
| 2. $S \rightarrow aSa$ ou bSb | (regra 2) | 2. $S \rightarrow aSa \mid bSb$ |

Por exemplo: $S \Rightarrow aSa \Rightarrow abSba \Rightarrow ab\lambda ba = abba$

Temos assim as produções da Gramática dos palíndromos

$$S \rightarrow a \mid b \mid \lambda ,$$

$$S \rightarrow aSa \mid bSb$$

ou

$$S \rightarrow aSa$$

$$S \rightarrow bSb$$

$$S \rightarrow a$$

$$S \rightarrow b$$

$$S \rightarrow \lambda$$

em que ‘|’ significa “ou” e S é o símbolo inicial.

$$G = (\{S\}, \{a, b\}, S, P)$$

Bibliografia

An Introduction to Formal Languages and Automata, Peter Linz, 3rd Ed., Jones and Bartlett Computer Science, 2001
(livro mais seguido na cadeira)

Models of Computation and Formal Languages, R. Gregory Taylor, Oxford University Press, 1998.

Introduction to Automata Theory, Languages and Computation, 2nd Ed., John Hopcroft, Rajeev Motwani, Jeffrey Ullman, Addison Wesley, 2001.

Elements for the Theory of Computation, Harry Lewis and Christos Papadimitriou, 2nd Ed., Prentice Hall, 1998.

Introduction to the Theory of Computation, Michael Sipser, PWS Publishing Co, 1997.